

دليل تنفيذ Create Category — خطوة بخطوة

اكتب كل ملف بيدك بالترتيب، وانسخ الكود من هنا

الفكرة: هنعمل موديول جديد اسمه `category` على نفس نمط باقي الموديولات (`user / brand`). كل ملف هتكتبه بيدك — الكود جاهز في الدليل ده. بعد كل خطوة خلّك مرتاح، ولو وقفت عند أي حاجة اسأل.

0. الأول: اتأكد إن السكيما جاهزة

التعديل ده اتعمل سابقًا واتفّح على البرانش. خذ بالك من شكل الـ `Category` في `prisma/schema`:

```
model Category {
  id          String      @id @default(uuid()) @db.Uuid
  brandId     String      @map("brand_id") @db.Uuid
  name        String      @map("category_name")
  imageUrl    String?     @map("image_url")
  media       Json
  ishidden    Boolean     @default(false)
  Products    Product[]
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @default(now())
  brand       Brand @relation(fields: [brandId], references: [id],
onDelete: Cascade)

  @@unique([brandId, name])
  @@map("categories")
}
```

• `brandId` — بيخزن البراند صاحب القسم (المفتاح الأجنبي).

• `@@unique([brandId, name])` — الاسم فريد ضمن البراند بس.

• media Json – حقل موجود قديمًا ومطلوب، فنبعت له {} وقت الإنشاء.

• لو مش متأكد إنه اتحفظ في الداتايز، شغل: `npx prisma db push`

Commit بتاع الخطوة دي: التعديل ده ارفع سابقًا بال `commit` ده:

```
git commit -m "feat(category): add brand relation to category schema"
```

مفيش حاجة تعمله هنا – بس اتأكد إن البرانش بتاعك عليه (شغل `git log --online` واتأكد إنه موجود).

1. اعمل الفولدر والملفات

خلي الاسم حرف صغير (مثل باقي الموديولات). اعمل فولدر جديد جوه `src/modules/`:

```
src/modules/category/  
├─ category.validators.ts  
├─ category.model.ts  
├─ category.service.ts  
├─ category.controller.ts  
└─ category.routes.ts
```

لو الفولدر القديم `Category` (بحرف كبير) موجود – اعمل له `rename` ل `category` أو احذفه. الفولدرات بتفرق في حرف صغير/كبير على لينكس.

Commit بتاع الخطوة دي: بعد ما تعمل الـ 5 ملفات (حتى لو فاضية) وحطها في

الفولدر:

```
git add src/modules/category git commit -m "feat(category): add
category module skeleton" git push
```

2. الخطوة الأولى – ملف الـ validators

ده بيستخدم zod للتحقق من المدخلات. بنلاقي الـ `brandId` جاي من الـ `URL params`، و `name` و `imageUrl` من الـ `body`. الميدلوير بتاع الـ `validate` في المشروع بيضيف `params` لـ `body`، فبيشتغلوا مع بعض في `schema` واحد.

```
src/modules/category/category.validators.ts
```

```
import { z } from "zod";

export const createCategorySchema = z.object({
  brandId: z.string().uuid({ message: "Invalid brand ID format" }),
  name: z.string().min(1, "Category name is required"),
  imageUrl: z.string().url("Invalid image URL format").optional(),
});
```

- `brandId` – لازم يكون UUID صالح (قائمة من الرابط).
- `name` – إجباري وأقل حاجة حرف واحد.
- `imageUrl` – اختياري، ولو جاي لازم يكون URL صالح.

Commit بتاع الخطوة دي: بعد ما تكتب الملف:

```
git add src/modules/category/category.validators.ts git commit
-m "feat(category): validate create category inputs" git push
```

3. الخطوة الثانية – ملف ال model

ده اللي بيتكلم مع قاعدة البيانات (Prisma). قبل ما تعمل create، لازم تتأكد إن الاسم مش مكرر ضمن البراند – عشان نرمي خطأ 409 لو مكرر.

```
src/modules/category/category.model.ts
```

```
import prisma from "../../../config/database";

class CategoryModel {
  async findByNameWithinBrand(brandId: string, name: string) {
    return prisma.category.findFirst({
      where: { brandId, name },
      select: { id: true },
    });
  }

  async create(brandId: string, name: string, imageUrl?: string) {
    return prisma.category.create({
      data: {
        brandId,
        name,
        media: {},
        ...(imageUrl ? { imageUrl } : {}),
      },
      select: { id: true },
    });
  }
}

export const categoryModel = new CategoryModel();
```

- findByNameWithinBrand – يفحص لو قسم بنفس الاسم موجود عند نفس البراند.
- create – ينشئ القسم ويرجع id بس.
- media: {} – الحقل المطلوب في السكيما، بنبعته JSON فاضي.
- ...(imageUrl ? { imageUrl } : {}) – لو imageUrl مش موجود، متضيفوش.

Commit بتاع الخطوة دي: بعد ما تكتب الملف:

```
git add src/modules/category/category.model.ts git commit -m  
"feat(category): add category model create methods" git push
```

4. الخطوة الثالثة – ملف ال service

ده "منطق الأعمال" – الترتيب مهم هنا: الأول البراند موجود؟ وبعدين إنت صاحبه؟ وبعدين الاسم مش مكرر؟

```
src/modules/category/category.service.ts
```

```
import { categoryModel } from "../category.model";  
import { brandModel } from "../brand/brand.model";  
import AppError from "../../../shared/errors/AppError";  
import { StatusCodes } from "http-status-codes";  
  
class CategoryService {  
  async createCategory(brandId: string, userId: string, role:  
string, name: string, imageUrl?: string) {  
    const brand = await  
brandModel.findBrandWithDocuments(brandId);  
    if (!brand) {  
      throw new AppError("Brand not found",  
StatusCodes.NOT_FOUND);  
    }  
  
    if (brand.userId !== userId && role !== "admin") {  
      throw new AppError("You are not authorized to create  
categories for this brand", StatusCodes.FORBIDDEN);  
    }  
  
    const existing = await  
categoryModel.findByNameWithinBrand(brandId, name);  
    if (existing) {  
      throw new AppError("Category name already exists in this  
brand", StatusCodes.CONFLICT);  
    }  
  }  
}
```

```

    }

    const category = await categoryModel.create(brandId, name,
imageUrl);
    return { id: category.id };
  }
}

export const categoryService = new CategoryService();

```

الفحص	لو فشل	سبب
البراند موجود؟	404 Not Found	الرابط فيه brandId مش موجود
صاحب البراند؟	403 Forbidden	مينفعش أي حد يضيف قسم لبراند مش بتاعه
الاسم مكرر؟	409 Conflict	اسم القسم موجود عنده بالفعل

ملاحظة: `brandModel.findBrandWithDocuments` موجودة جاهزة في موديول الـ `brand` — بنعيد استخدامها بدل ما نكتب `query` ثاني.

Commit بتاع الخطوة دي: بعد ما تكتب الملف:

```
git add src/modules/category/category.service.ts git commit -m
"feat(category): add create category business logic" git push
```

5. الخطوة الرابعة – ملف الـ controller

ده بياخد الـ `request`، بينادي الـ `service`، ويرجّع الرد. بيستخدم `asyncHandler` زي باقي المشروع عشان الأخطاء تتروح للـ `error handler` لوحدها.

```
src/modules/category/category.controller.ts
```

```

import { Request, Response } from "express";
import { categoryService } from "../category.service";
import { StatusCodes } from "http-status-codes";
import { asyncHandler } from "../../../shared/utils/asyncHandler";
import AppError from "../../../shared/errors/AppError";

export const createCategory = asyncHandler(async (req: Request, res: Response) => {
  const { brandId } = req.params as { brandId: string };
  const { name, imageUrl } = req.body as { name: string; imageUrl?: string };
  const user = (req as any).user;

  if (!user?.id) {
    throw new AppError("Not authenticated", StatusCodes.UNAUTHORIZED);
  }

  const result = await categoryService.createCategory(brandId, user.id, user.role, name, imageUrl);

  res.status(StatusCodes.CREATED).json({
    success: true,
    message: "Category created successfully",
    data: result,
  });
});

```

• req.params.brandId — من الرابط.

• (req as any).user — جاي من ميديوير ال authenticate (فيه id و role).

• بيرجع 201 مع data.id — زي ما التاسك طالب بالظبط.

Commit بتاع الخطوة دي: بعد ما تكتب الملف:

```

git add src/modules/category/category.controller.ts git commit
-m "feat(category): add create category controller" git push

```

6. الخطوة الخامسة – ملف ال routes

بیمرر الطلب على ال middlewares بالترتيب ثم ال controller.

```
src/modules/category/category.routes.ts
```

```
import { Router } from "express";
import { validate } from "../../../shared/middleware/validate";
import authenticate from "../../../shared/middleware/authenticate";
import { authorize } from "../../../shared/middleware/authorize";
import { createCategory } from "../category.controller";
import { createCategorySchema } from "../category.validators";

const router = Router();

router.post("/:brandId/categories",
  authenticate,
  authorize("brand"),
  validate(createCategorySchema),
  createCategory,
);

export default router;
```

لاحظ إنا كتبنا `/:brandId/categories` من غير `/brands` في الأول – لأن
ال router ده هيتركب على `/api/v1/brands` في الخطوة الجاية، فالمجموع يبقى `/api/v1/brands/:brandId/categories`

Commit بتاع الخطوة دي: بعد ما تكتب الملف:

```
git add src/modules/category/category.routes.ts git commit -m  
"feat(category): add create category routes" git push
```

7. الخطوة السادسة – التركيب في server.ts

افتح src/server.ts وعمل تعديلين:

أولاً: ضيف السطر ده في ال imports (بعد سطر ال brandRouter):

```
import categoryRouter from "../modules/category/category.routes";
```

ثانياً: ضيف السطر ده بعد سطر ال brands (لأن الرابط كله بيبدأ بـ /brands/):

```
app.use("/api/v1/brands", categoryRouter);
```

شكل ال routes النهائي:

```
app.use("/api/v1/auth", authRouter);
app.use("/api/v1/brands", brandRouter);
app.use("/api/v1/users", usersRouter);
app.use("/api/v1/brands", categoryRouter); // ← ضيف ده
```

خد بالك من نقطتين:

1. ال mount على /api/v1/brands – مش /api/v1/categories – عشان الرابط الكامل يطلع مطابق للتاسك.
2. اتأكد إن مفيش userRouter مستورد مرتين في ال imports (كان في نسخ قديمة مكرر).

Commit بتاع الخطوة دي: بعد ما تعدل ال server.ts:

```
git add src/server.ts git commit -m "feat(category): mount
category routes" git push
```

8. الخطوة الأخيرة – تحقق إن كل حاجة سليمة

شغل دول عشان تتأكد إن الكود بيتجمع من غير أخطاء:

```
npm run build
```

```
npx prisma validate
```

ملاحظة: ممكن يطلعلك أخطاء في موديول الـ `brand` (مش بتاعنا) لأن الكود القديم على البرانش مش متوافق مع السكيما ومستخدم `axios` مش مثبت. اللي يهملك: مفيش أخطاء في ملفات الـ `category`.

9. جرب الـ endpoint

بعد ما تشغل السيرفر (`npm run dev`)، اختبر في Postman:

#	السيناريو	النتيجة
1	POST بدون توكن	401
2	توكن بس المستخدم <code>role = user</code>	403
3	براند بيضيف اسم موجود عنده	409
4	براند بيضيف اسم جديد	201 بيديك <code>data.id</code>
5	<code>brandId</code> مش موجود	404

```
POST /api/v1/brands/:brandId/categories Headers: Authorization: Bearer <accessToken> Body (JSON): { "name": "Women's Wear", "imageUrl": "https://cdn.example.com/category.png" }
```

10. ملخص كل ال Commits — بأي ترتيب

ده ملخص ال commits اللي هتعملهم على `feature/createCategory` ، كل واحد بعد ما تخلص خطوته:

#	بعد ما تخلص	اسم ال commit
0	تعديل السكيما (مفعول من قبل)	<code>feat(category): add brand relation to category schema</code> مرتّب
1	عمل الفولدر وال 5 ملفات الفاضية	<code>feat(category): add category module skeleton</code>
2	كتابة validators	<code>feat(category): validate create category inputs</code>
3	كتابة model	<code>feat(category): add category model create methods</code>
4	كتابة service	<code>feat(category): add create category business logic</code>
5	كتابة controller	<code>feat(category): add create category controller</code>
6	كتابة routes	<code>feat(category): add create category routes</code>
7	التعديل في server.ts	<code>feat(category): mount category routes</code>
8	بعد نجاح ال build	<code>chore(category): build passes</code> (اختياري)

معنى ال commit message:

feat = ميزة جديدة | fix = إصلاح خطأ | chore = صيانة/تنظيف

(category) = الموديول أو الجزء المتأثر

الكلمة بعدها = وصف مختصر للي تعمل

مثال: feat(category): add create category routes

الترتيب كله في جملة: validators (تحقق) → model (الداتايز) → service

(المنطق) → controller (الرد) → routes (الوصل) → server.ts (التركيب) — وبعد

كل ملف commit ثم push.